

Lumos: Performance Characterization of WebAssembly as a Serverless Runtime in the Edge-Cloud Continuum

Cynthia Marcelino

TU Wien
Wien, Austria
c.marcelino@dsg.tuwien.ac.at

Thomas Werner Pusztai

TU Wien
Wien, Austria
t.pusztai@dsg.tuwien.ac.at

Noah Krennmair

TU Wien
Wien, Austria
e12023994@student.tuwien.ac.at

Stefan Nastic

TU Wien
Wien, Austria
snastic@dsg.tuwien.ac.at

Abstract

WebAssembly has emerged as a lightweight and portable runtime to execute serverless functions, particularly in heterogeneous and resource-constrained environments such as the Edge-Cloud Continuum. However, the performance benefits versus trade-offs remain insufficiently understood. Current state-of-the-art performance characterizations either focus on isolated WebAssembly benchmarks or fail to address workload diversity, such as compute-intensive and data-intensive scenarios, in dynamic and heterogeneous environments such as the Edge-Cloud Continuum. Therefore, in this paper, we present Lumos, a performance model and benchmarking tool for characterizing serverless runtimes. Lumos identifies workload, system, and environment-level performance drivers in the Edge-Cloud Continuum. We benchmark state-of-the-art containers and the Wasm runtime in interpreted mode and with ahead-of-time compilation. Our performance characterization shows that AoT-compiled Wasm images are up to 30x smaller and decrease cold-start latency by up to 16% compared to containers, while interpreted Wasm suffers 30–55x higher warm latency and 7–10x I/O-serialization overhead.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; • **General and reference** → **Measurement**; • **Software and its engineering** → **Software performance**.

Keywords

Serverless computing, FaaS, Data transfer, WebAssembly

ACM Reference Format:

Cynthia Marcelino, Noah Krennmair, Thomas Werner Pusztai, and Stefan Nastic. 2025. Lumos: Performance Characterization of WebAssembly as a Serverless Runtime in the Edge-Cloud Continuum. In *The 15th International Conference on the Internet of Things (IOT 2025)*, November 18–21, 2025, Vienna, Austria. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3770501.3770515>



This work is licensed under a Creative Commons Attribution 4.0 International License. IOT 2025, Vienna, Austria

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1595-2/25/11
<https://doi.org/10.1145/3770501.3770515>

1 Introduction

Serverless computing provides infrastructure management, elastic scaling, and on-demand resource allocation. In serverless computing, user code is deployed as small, short-lived functions that respond to events, scaling up or down according to demand [7, 22, 36]. Recently, Wasm has emerged, offering serverless functions, strong isolation, and decreased cold starts. In Wasm-based functions, the code is compiled into small binary modules, which are executed within a lightweight, memory-safe, and secure sandbox, also known as the Wasm VM, enabling cross-platform portability while maintaining minimal overhead [8, 11, 30]. Moreover, Wasm follows a deny-by-default principle, which means Wasm VMs do not have access to the host unless explicitly allowed. To access host features such as network interfaces and filesystem, Wasm VMs leverage the WebAssembly System Interface (WASI) [9, 15, 38].

Wasm enables next-generation serverless computing by offering a portable, secure, and lightweight execution model, making it well-suited for heterogeneous, resource-constrained environments such as the Edge-Cloud Continuum [11, 25, 27, 30]. However, its performance characteristics in the Edge-Cloud Continuum remain insufficiently understood. Existing performance characterization of Wasm includes: (a) *Standalone Runtime Analysis* [12, 13, 17, 29, 40] focus on low-level Wasm runtime behavior relative to native execution, typically measuring execution latency, instruction count, and CPU and memory usage. However, they do not effectively evaluate the performance of Wasm applications when deployed as OCI bundles, nor compare against Docker containers, which is the state-of-the-art runtime execution in the Edge-Cloud Continuum. Understanding how Wasm-based runtimes perform in this context is essential for evaluating the potential overhead in serverless platforms that rely on container-based isolation. (b) *Serverless Runtimes* [15, 18, 21, 28] measure cold start latency, execution time, and warm start performance of functions deployed in serverless environments. However, they do not consider the impact of I/O and (de)serialization overhead, particularly in Wasm runtimes that use WASI for networking and file access. Thus, it limits a detailed identification of performance bottlenecks in serverless functions, which typically rely on remote services for data exchange.

While Wasm promises near-native speed, it is essential to understand the benefits and tradeoffs associated with it in order to build smart systems to optimize performance. Therefore, in this paper, we propose Lumos, a performance model and framework

that enables performance characterization of serverless functions, leveraging multiple serverless runtimes. Lumos enables users to evaluate serverless functions with different workload characteristics, such as compute and data-intensive, as well as measure data IO and serialization, which incurs a great impact on the performance of serverless functions in the Edge-Cloud Continuum [1, 19, 26].

We summarize our contributions as follows:

- *Lumos: A Performance Model* for serverless functions in the Edge-Cloud Continuum. We identify primary performance factors across workload, system, and environment. The Lumos model captures the influence of these dimensions on Service Level Objectives (SLOs) such as latency, throughput, and cost. Thus, enabling the identification of performance bottlenecks in the Edge-Cloud Continuum.
- *A benchmarking tool for evaluating serverless runtimes*. Lumos benchmark tool supports reproducible experiments across cold and warm starts, diverse workloads such as compute and data-intensive, and varying concurrency levels in heterogeneous environments;
- *A performance characterization of WebAssembly as serverless runtime*, examining their execution models, cold starts, isolation mechanisms, and integration within the serverless platforms. Our results show that Wasm Ahead-of-Time (AoT) images are up to 30x smaller and decrease cold-start latency by up to 16% compared to containers, while Wasm interpreted suffers 30–55x higher warm latency and 7–10x I/O-serialization overhead.

2 Research Challenges

In this section, we identify the following research challenges of Serverless function performance in the Edge-Cloud Continuum.

RC-1: How do workload, system, and environmental factors influence the performance of serverless functions across the Edge-Cloud Continuum? Serverless function performance typically relies on Service Level Objectives (SLOs) such as end-to-end execution time, cost, and throughput to evaluate its performance [32]. However, SLOs are affected by various interdependent factors that range from business logic to the underlying systems and environment. For example, workload characteristics such as data IO can impact function execution time by up to 95% [1, 19]. Additionally, inefficient system-level mechanisms, such as autoscaling and scheduling, can cause further delays [34]. Environmental factors such as device heterogeneity and network congestion can cause increased execution queue and/or failures [33]. Therefore, a performance model that captures broader influencing factors is essential for identifying performance bottlenecks and creating effective solutions to address them.

RC-2: How do serverless runtimes impact function performance in the Edge-Cloud Continuum? Wasm provides advantages such as compact binary size, which contributes to reduced cold start latency compared to container-based runtimes. However, Wasm executes within a sandboxed environment with linear memory and no direct access to host resources, relying on the WASI to enable interactions with the host for filesystem and network access, introducing

overhead, particularly in data-intensive applications [21, 25]. Furthermore, while Wasm interpreter-based execution ensures portability across platforms, it incurs higher runtime overhead compared to AoT and Just-in-Time (JIT) compilation [15, 17]. In contrast, container-based runtimes benefit from direct host integration, relying on Linux namespaces and cgroups to enforce isolation. While container isolation provides portability and scalability, it potentially exposes both the host and the functions to threats posed by untrusted tenants [8, 10]. Therefore, it is crucial to understand the factors that influence serverless function performance under varying workloads, systems, and environmental characteristics.

RC-3: How to characterize serverless function performance under different workload characteristics and heterogeneous environments, such as the Edge-Cloud Continuum? Existing performance characterization either focuses on standalone Wasm runtimes or serverless functions without capturing all influencing performance characteristics of the Edge-Cloud Continuum [15, 17, 28, 40]. Evaluating serverless runtimes requires the ability to simulate concurrency, diverse workload characteristics (e.g., compute- vs. data-intensive), cold vs. warm execution, and BaaS access. Additionally, fine-grained telemetry must be collected to isolate compute, I/O, and serialization time, and to attribute resource usage to specific functions. Therefore, a modular and extensible benchmarking that integrates multiple runtimes, enabling reproducible experiments with precise resource attribution, is essential for a precise performance characterization.

3 Serverless Performance Model for the Edge-Cloud Continuum

Typically, the performance of serverless functions is measured by SLOs, such as end-to-end execution time, throughput, costs, and error rate. However, the performance of serverless functions can be influenced by various factors. To effectively analyze this variability, we introduce a performance model that categorizes the main performance factors into: workload, system, and environment, as shown in Fig. 1. Each dimension represents a unique influence layer in the function lifecycle (Fig. 2) that affects the overall function performance. Lumos' performance model facilitates the profiling of serverless functions' performance, allowing for precise benchmarking.

3.1 Workload Performance Factors

Function Heterogeneity. Serverless functions display a variety of workload characteristics, such as compute-intensive and data-intensive tasks that involve significant data movement. Lumos offers benchmarking capabilities, enabling users to evaluate how the performance of runtimes scales with different application variations, providing users with insights, as certain runtimes may perform better with specific workloads, such as CPU-bound tasks, while struggling with others, such as I/O operations that involve a high overhead from system calls.

Workflow Dependencies. Serverless workflows often comprise interconnected functions that exchange ephemeral data [27]. Performance issues in one function in the workflow may impact the performance of the entire workflow, as subsequent functions depend on outputs from previous functions.

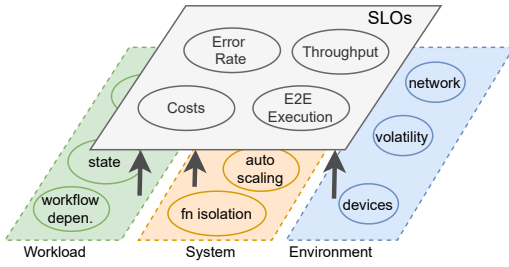


Figure 1: Serverless Performance Model for the Edge-Cloud Continuum, highlighting performance factors: workload, system, and environment.

Function State and BaaS. Typically, serverless functions are stateless and rely on external remote services to store function state. These additional services used by the functions are referred to as Backend-as-a-Service (BaaS). The integration of BaaS goes beyond just state management; serverless functions also utilize BaaS to perform more complex tasks, such as ETL and AI inference. While BaaS accelerates development and promotes modularity, it introduces external dependencies and increases latency and network overhead [1, 19, 26].

Serialization Overhead. Serverless functions are typically written in high-level languages such as NodeJS, Python, and Java and exchange complex data structures, which incur high (de)serialization costs [4, 6, 23]. For instance, AWS@Lambda has reported over 67% of functions running in less than 20ms. On the other hand, the (de)serialization process of a complex Python object of 3MB takes around 10 ms. Therefore, as serverless functions are short-lived, additional overhead with (de)serialization can significantly impact the execution time [19, 23].

3.2 System Performance Factors

Performance Isolation. In a multi-tenant environment, such as serverless, performance isolation ensures that one function’s behavior does not impact others. Poor isolation performance results in noisy neighbor effects, where co-located functions contend for CPU, memory, or network bandwidth. For instance, a compute-bound function may monopolize CPU time, leading to scheduling delays for co-resident functions. Similarly, shared memory bandwidth and last-level cache can be saturated by data-intensive functions, causing delays on neighbor functions [42–44].

Function Isolation. Serverless functions are tiny pieces of code wrapped in bundles and operated by serverless platforms. Typically, serverless functions are executed in a multi-tenant environment, sharing host hardware and OS. To ensure that functions do not interfere with each other, they are isolated using different mechanisms, such as Docker containers, unikernels, and Wasm VMs [8, 9, 38]. Container-based isolation offers native execution of functions but increases cold starts as functions must provision their file system and setup cgroups. WasmVM offers a lightweight isolation, but it introduces overhead due to WASI communication [11, 21, 25].

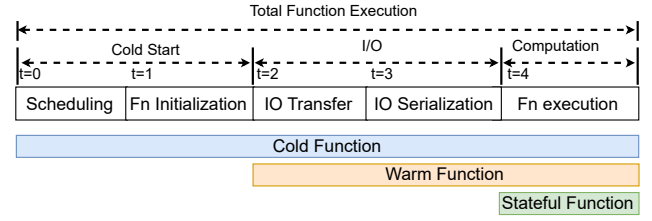


Figure 2: Serverless function lifecycle highlighting cold, warm, and stateful functions

Function Invocation. Typically, serverless function execution is characterized by dynamic invocation behaviors such as bursty invocations, scheduled triggers, and multi-tenant requests [6]. Bursty workloads can temporarily overload the platform, leading to request queuing, increased cold starts, and resource contention. Additionally, uneven invocation rates can create a concurrency imbalance, where dominant functions consume shared resources, ultimately increasing the latency of identical functions up to 338% [41]. To maintain performance under diverse workloads, platforms must balance their system-level mechanisms, such as concurrency limits and scaling thresholds.

Auto Scaling and Cold Starts. In serverless computing, functions are not continuously running and are instantiated only upon demand. When no active instance is available, the platform must provision the necessary runtime environment, a process known as a cold start. To manage demand, auto scalers dynamically adjust the number of function instances based on incoming traffic [7, 31, 33]. While this elasticity improves resource utilization, it is sensitive to feedback lag. Bursty workloads may not be reflected in time, potentially causing the auto scaler to react too late or scale inadequately [39].

3.3 Environmental Performance Factors

Nodes and Devices Capabilities. The performance of serverless functions is influenced by the characteristics of their execution environments. Specifically, in the Edge-Cloud Continuum, which can vary due to device heterogeneity, including differences in CPU architecture, memory capacity, and GPU capabilities, functions that require specific hardware may experience degraded performance if deployed on under-provisioned nodes. Similarly, memory-bound functions can suffer from allocation failures on low-memory edge devices, leading to increased latency or function retries.

Network Variability. Typically, serverless functions rely on network connectivity to access data, invoke downstream services, or communicate with external BaaS components. In the Edge-Cloud Continuum, function performance is affected by variable latency, fluctuating bandwidth, and intermittent connectivity [24]. High jitter or packet loss can lead to retries, timeout errors, and degraded throughput. Therefore, network performance directly impacts total execution time, particularly for data-intensive functions.

Devices Volatility. In resource-constrained environments such as Edge-Cloud Continuum, devices might be battery-powered or change positions, making nodes’ availability unpredictable and

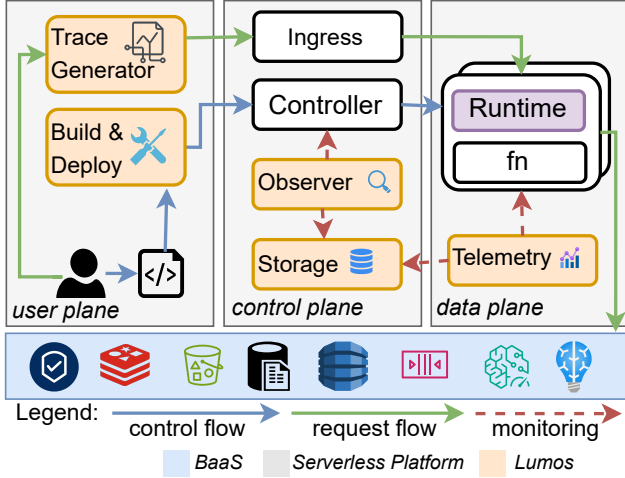


Figure 3: Lumos Architecture Overview

consequently leading to load shifts to other nodes in the cluster. This instability leads the platform to reschedule functions, which may incur cold starts, increased communication overhead, and consequently impact overall performance.

4 Lumos Architecture Overview and Performance Metrics

In this section, we present Lumos architecture overview and performance metrics that enable serverless performance characterization.

4.1 Lumos Architecture Overview

Lumos provides an end-to-end benchmarking tool across three planes, as shown in (Fig. 3). In the *user plane*, functions are prepared and reproducible traces are generated; in the *control plane*, scheduling and routing are observed and control metadata is persisted; and in the *data plane*, state I/O is instrumented to capture compute and BaaS effects. This three-plane decomposition provides detailed performance insights while ensuring both portability and low overhead. It maintains a clear separation of concerns, allowing workloads to evolve independently from the serverless platform. Additionally, it enables seamless runtime changes (such as Wasm or containers) without necessitating changes to the measurement stack. As each step in a function’s execution presents unique trade-offs (e.g., cold starts and BaaS access latency), Lumos profiles these function execution steps to identify bottlenecks and measure their impacts across serverless workloads.

Built-in Functions. Lumos includes a set of representative serverless workloads designed to capture diverse performance behaviors. These functions cover both data-intensive and compute-bound workloads. They are used to evaluate how different serverless runtimes handle varying workload characteristics. Table 1 provides an overview of the selected workloads and their corresponding profiles.

4.1.1 Lumos Components. Lumos is composed of six components that span from control to data plane and user plane.

Table 1: Description of Lumos Built-in Functions

Function	Description	High CPU	High I/O
audio-generation	Generate audio	✓	✓
fuzzy-search	Fuzzy text search	✓	✓
language-detection	Text language detect	✓	✓
encrypt-message	Encrypt data	✓	✗
decrypt-message	Decrypt data	✓	✗
fibonacci	Fibonacci sequence	✓	✗
prime-numbers	Prime number calculation	✓	✗
mandelbrot-bitmap	Generate Mandelbrot bitmap	✓	✓

Build and Deploy. Lumos provides a modular build pipeline that supports reproducible and portable benchmarking. The build pipeline enables serverless functions to be compiled into Docker containers using a base image or Wasm modules in interpreted or AoT compilation. The deployment pipeline integrates seamlessly with the serverless platform, such as Knative. Lumos provides deployment templates that enable consistent rollout across runtime configurations while isolating experiment conditions such as storage type, payload size, and function parameters. Based on the user input, Lumos selects the runtime, selection via the orchestrator deployment config, i.e., `.spec.template.spec.runtimeClassName`.

Trace Generator. It creates synthetic invocation traces based on configurable profiles. The generator can simulate concurrency levels, delayed invocations, and realistic bursty invocations.

Observer. It intercepts and timestamps key lifecycle events (e.g., ingress, data IO, serialization, function ready, response sent). It aggregates metrics from runtime logs, platform APIs, and custom probes injected into the function code to aid the data IO and serialization identification. Additionally, the observer collects the function runtime ID and its PID to enable precise correlation across logs and telemetry streams.

Storage. It stores collected metrics, logs, and structured traces. The storage enables future integration with observability tools such as Prometheus, Loki, or OpenTelemetry for unified analysis and monitoring across experiments.

Telemetry. It collects fine-grained resource metrics (e.g., CPU times, memory usage, I/O operations) via cgroup, and function runtime, thus allowing the identification of resource usage for specific functions. It tracks usage to specific processes using the container IDs, enabling per-request resource attribution.

4.2 Performance Metrics

In this section, we define the metrics provided by Lumos that enable granular performance evaluation of serverless functions. Each metric captures a distinct aspect of the function lifecycle, shown in Fig. 2, from invocation to execution and resource consumption, enabling detailed analysis across different serverless runtimes.

Total Function Execution Time. It reflects the combined impact of platform delays such as auto scaling, scheduling, request forwarding, as well as function logic, I/O operations, and other external dependencies. It is the most visible metric to users and essential for evaluating non-functional requirements such as SLO compliance.

I/O Latency. It shows the time to access external services such as databases and object stores. In serverless workflows, frequent access to BaaS can dominate execution time, especially in data-intensive applications or workflows with function chains.

Serialization. It accounts for the cost of encoding and decoding data between functions or when communicating with external services. Complex data structures incur significant (de)serialization overhead, which impacts short-lived functions significantly.

Compute Time. It measures the function computation without cold start initialization, IO, and serialization overhead. The compute time isolates the pure logic execution phase to enable runtime efficiency evaluation, highlighting performance differences in compute-bound workloads across serverless runtimes.

Image Size. Function image size impacts the pull time, which is necessary during cold start. Specifically, in the congested network, larger images may suffer increased pull times, impacting overall function execution time. On the other hand, smaller images, especially in Wasm runtimes, can significantly reduce cold starts.

Coldstart Time. It measures the time between a request’s arrival on the platform and the function becoming ready for execution. It includes image loading, scheduling, and environment provisioning.

CPU Usage. It provides fine-grained CPU time consumed during function execution. It is essential for profiling compute-bound functions and assessing runtime efficiency across different execution environments.

Memory Usage. It measures the memory footprint of the function. It is particularly important for data-intensive functions in constrained environments such as edge nodes, where memory limits may affect function placement.

5 Implementation

Lumos is implemented as a modular benchmarking framework designed to evaluate the performance of serverless runtimes such as Wasm and container-based serverless functions across heterogeneous edge platforms. Lumos components are implemented in Bash, Python, and Rust, and support full automation of build, deployment, benchmarking, and telemetry collection pipelines. It is published as an open-source framework and is available on GitHub¹

Configurability and Logging. All configuration settings for the benchmarking pipeline are centralized in a global file, including properties such as architecture targets, image registry endpoints, and run. Function-specific parameters such as input sizes and storage types are encoded via JSON payloads. During runtime, each function communicates with its backend-as-a-service storage based on runtime-configurable parameters passed via JSON payloads. This design allows precise analysis of the impact of data and compute-intensive workloads on cold and warm starts.

Functions and Workloads. The Lumos built-in workloads include eight serverless functions implemented in Rust, selected to represent data-intensive and compute-intensive functions. Each function exposes a single POST endpoint and supports native and Wasm

¹<https://github.com/polaris-slo-cloud/lumos>

Table 2: Payload size groups

Function	Group Size 1	Group Size 2	Group Size 3
audio-generation	512 Kb	1024 Kb	2048 Kb
fuzzy-search	512 Kb	1024 Kb	2048 Kb
language-detection	512 Kb	1024 Kb	2048 Kb
encrypt-message	512 Kb	1024 Kb	2048 Kb
decrypt-message	512 Kb	1024 Kb	2048 Kb
fibonacci	10,000 int	100,000 int	1,000,000 int
prime-numbers	10,000 int	100,000 int	1,000,000 int
mandelbrot-bitmap	512 Kb	1024 Kb	2048 Kb

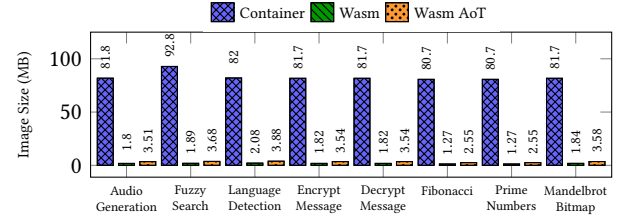


Figure 4: Image sizes of serverless functions

compilation targets via conditional Cargo configurations. Functions are invoked using structured payloads, with parameters controlling function behavior and input size. Table 1 summarizes each workload and resource profile.

6 Evaluation

The goal of our experiments is to characterize performance under varying input sizes and concurrency levels and assess resource efficiency. We benchmark the serverless functions spanning data and compute-intensive workloads as described in Table 1. Input sizes range from 512KB to 2MB for data-intensive and up to 2M integer as described in Table 2, reflecting structured data commonly exchanged between serverless functions in edge-cloud workflows [6]. We conduct both sequential executions, as well as parallel executions, aligning with real-world serverless workflows, as outlined in [14].

6.1 Experimental Setup

Our experiments are conducted on a heterogeneous edge cluster to emulate realistic resource-constrained environments. Our testbed includes two Raspberry Pi Model 5B with 4x2.4 GHz CPUs with 16 GB and 8 GB RAM, respectively, and two Raspberry Pi 4B with 4x1.8 GHz CPUs and 4 GB RAM. All devices have 64 GB storage and 100 Mbps network links. As an orchestration tool, we use MicroK8s, Knative as a serverless platform, and Redis as storage. For the Wasm runtime management, we leverage the Kwasm operator. Each serverless function is deployed as a Knative service. All serverless functions are implemented in Rust. Telemetry for CPU and memory usage is collected using telemd [5], which runs on each node. Each experiment is repeated 10 times, and mean values are reported to ensure statistical significance.

6.2 Baselines

For our baselines, we use RunC as the container runtime for native functions, and WasmEdge as the WebAssembly runtime, evaluated

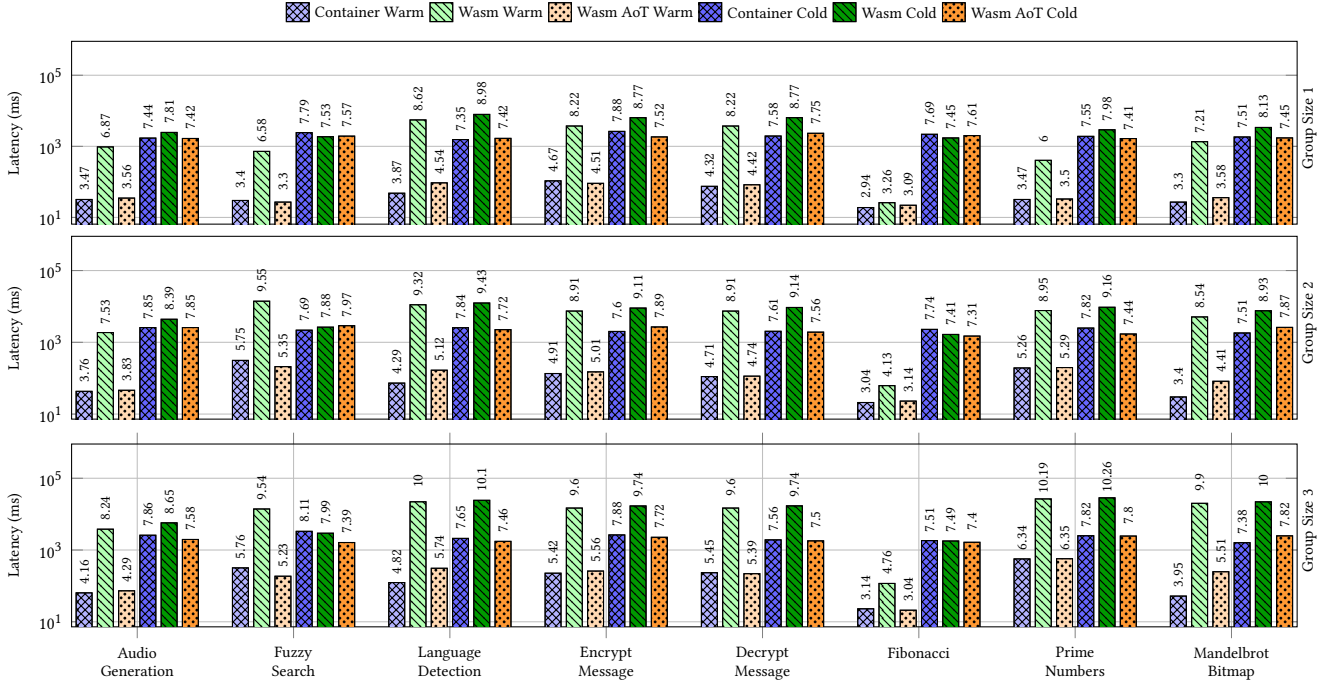


Figure 5: Execution time comparison across I/O- and CPU-intensive functions for varying input sizes and group sizes.

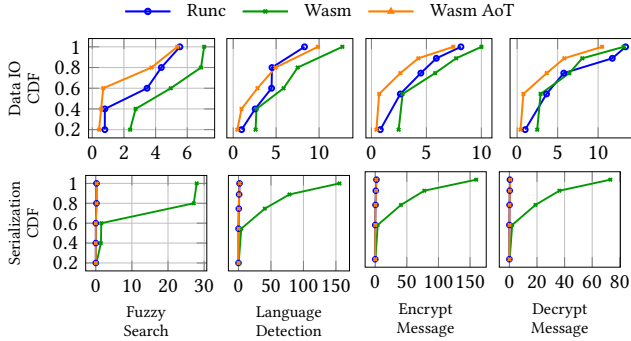


Figure 6: CDF Latency IO and Serialization Overhead

in both interpreted and AoT modes. While we evaluate functions on the WasmEdge runtime, Lumos is designed to be extensible and modular, allowing seamless integration of alternative WebAssembly runtimes such as Wasmtime and Spin via Kwasm.

6.3 Image Size Results

In this experiment, we compare the image sizes of deployed serverless functions across the three runtime configurations: Docker Container, WasmEdge (interpreted), and WasmEdge AoT. Image size is a key factor in cold start performance, deployment latency, and storage overhead, particularly relevant in edge environments where

bandwidth and storage are constrained. Each workload is packaged using identical source code and dependencies. While containers have a minimal base image (i.e., `debian:bullseye-slim`), WebAssembly images have a single binary packed from scratch.

Fig. 4 presents the resulting image sizes. Docker Container images produce significantly larger images, averaging over 80MB, AoT-compiled binaries ranging from 2.5 to 3.9 MB, and interpreted Wasm binaries range from 1.2 to 2MB. Wasm shows a reduction in image size up to 30x compared to containers, highlighting a major advantage of Wasm for scenarios with limited network capacity.

Finding 1: Wasm reduces image size by up to 30x compared to containers, highlighting its benefit in scenarios with limited network.

6.4 Function Execution Time Results

Fig. 5 compares the end-to-end function execution time (in milliseconds) of multiple serverless functions under warm and cold starts across three group sizes, reflecting increasing input payloads as defined in Table 2. Across all group sizes, Container and WasmEdge AoT show lower latency under warm starts, while interpreted WasmEdge exhibits significantly higher latency, particularly in compute-intensive functions such as Prime Numbers and Mandelbrot Bitmap. For cold starts, WebAssembly AoT performs competitively with containers, especially for smaller group sizes, though performance diverges as the input size increases. Interpreted WasmEdge suffers from consistently high cold start latency, with values exceeding 24 seconds in large-input compute-intensive workloads.

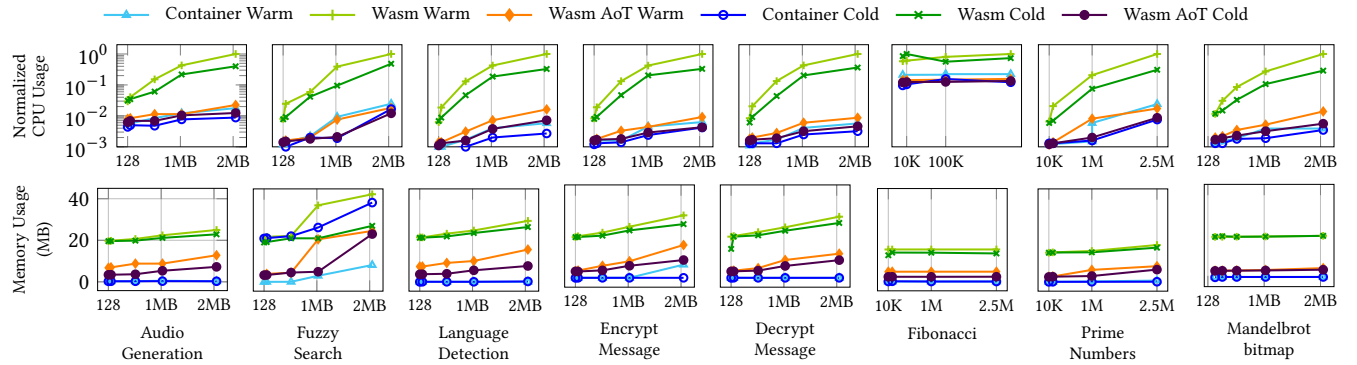


Figure 7: Resource Usage

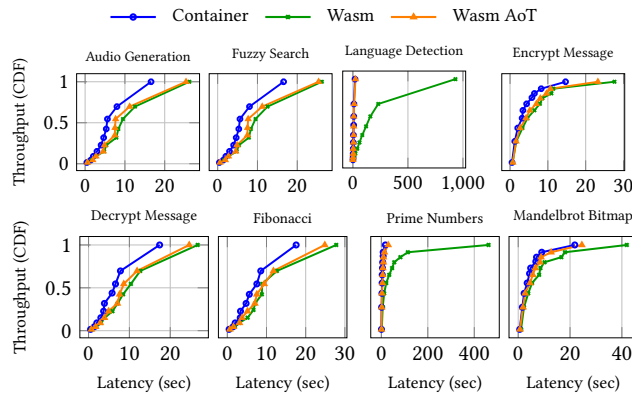


Figure 8: CDF throughput over latency across increasing concurrent requests. The plots compare the scalability behavior of three serverless runtimes Container, WasmEdge (interpreted), and WasmEdge AoT.

Finding 2: Warm Functions: Wasm AoT shows up to 1.1x the latency of containers across both compute- and data-intensive functions. Wasm interpreted is up to 30x slower for compute-intensive functions and up to 55x slower for data-intensive functions.

Finding 3: Cold Functions: Wasm AoT shows up to 16% lower latency than containers, particularly in data-intensive functions. Wasm interpreted remains up to 1.7x slower for compute-intensive and up to 2.9x slower for data-intensive functions.

6.5 IO and Serialization Results

As Wasm showed increased execution time, particularly for data-intensive functions (Section 6.4), in this experiment, we isolate the impact of I/O and serialization overhead to understand how data handling contributes to the overall performance of different runtimes. Fig. 6 shows the normalized data retrieval and serialization overhead across increasing payload sizes. Wasm AoT introduces up to 2x I/O overhead compared to containers, while Wasm interpreted incurs up to 7x higher I/O overhead, especially for large payloads.

During serialization, Wasm AoT maintains close-to-native serialization performance, with only up to 2x overhead over containers, while Wasm interpreted shows serialization costs exceeding 10x for larger payloads.

Finding 4: I/O and serialization significantly impact performance in Wasm runtimes. Wasm AoT incurs up to 2x in I/O and serialization overhead, while Wasm interpreted shows up to 7x higher I/O overhead and over 10x serialization overhead compared to containers.

6.6 Resource Usage Results

Fig. 7 compares normalized CPU and memory usage under varying input sizes, across different serverless runtimes, Container, WasmEdge (interpreted), and WasmEdge AoT runtimes. Across all workloads, containers consistently exhibit the lowest CPU and memory consumption. For compute-intensive functions such as Fibonacci and Prime Numbers, WasmEdge incurs 5–20x higher CPU usage than containers at larger input sizes, while AoT reduces the overhead but remains up to 3x higher. Memory usage patterns reveal similar trends, where interpreted WasmEdge consumes up to 42 MB for Fuzzy Search and Language Detection, compared to under 1 MB for containers. AoT shows a lower memory usage ranging from 5 to 16 MB.

Finding 5: Wasm shows up to 20x higher CPU usage and 40x more memory, for both compute- and data-intensive workloads.

6.7 Scalability Results

Fig. 8 presents normalized request per second (RPS), i.e., throughput over latency for increasing concurrency levels across eight serverless workloads ranging from 1 to 200 RPS. The results compare three runtime configurations: Container, WasmEdge (interpreted), and WasmEdge AoT. Containers maintain the most stable performance under load, sustaining throughput even as latency exceeds 10s. Data-intensive functions such as Audio Generation and Fuzzy Search show up to 50% lower latency in Container compared to WasmEdge at peak load. Compute-intensive workloads such as Prime Numbers and Mandelbrot Bitmap reveal even greater differences: WasmEdge experiences high degradation up to 461s latency,

while workloads running on containers complete execution below 18s. AoT compilation improves WasmEdge’s performance under moderate load, but high concurrency still leads to substantial latency growth, particularly in mixed workloads such as Language Detection, where interpreted WasmEdge peaks at over 930s.

Finding 6: Workloads running Containers consistently outperform WebAssembly (interpreted), achieving up to 27x lower latency and 2x higher throughput under high concurrency. WasmEdge AoT improves over interpreted mode but still degrades under high load. **Compute-intensive workloads** show the largest gaps, with WebAssembly with quick performance degradation. **Data-intensive functions** show a fair benefit from WebAssembly AoT compilation, which narrows the performance gap to containers but still lags under high concurrency.

7 Related Work

Serverless Function Benchmarks. SeBS [2, 3, 37] introduces a benchmarking suite for evaluating the performance of serverless workflows across cloud platforms, measuring function execution time, orchestration overhead, cold start frequency, scalability, and cost. SeBS provides a platform-independent methodology to compare serverless workflow performance. However, SeBS does not support multiple serverless runtimes, which is crucial for resource-constrained environments such as the Edge-Cloud Continuum. XFBench [20] evaluates serverless platforms using customizable workflows, workloads, and functions. It generates and deploys function DAGs, invoking workloads via JMeter, and collecting telemetry for detailed analysis across multiple platforms. XFBench enables reproducible benchmarking and helps developers analyze FaaS suitability, optimize workflow composition, and identify cloud platform bottlenecks. EdgeFaaS Bench [35] evaluates the performance of serverless computing on edge devices with heterogeneous hardware capabilities, such as CPU and GPU. EdgeFaaS Bench captures metrics such as cold/warm start times, concurrent execution overheads, and system-level utilization. However, EdgeFaaS Bench does not differentiate the impact of workload characteristics, such as compute and data-intensive. Thus, it limits the user’s ability to optimize the workflow based on the workload characteristics.

WebAssembly Benchmarks. WABench [40] presents a comprehensive characterization of predominant standalone WebAssembly runtimes, including Wasmtime, Wasmer, WAVM, Wasm3, and WAMR. While WABench focuses on low-level runtime behavior (e.g., performance, instruction count, cache usage, memory overhead) using synthetic benchmarks and full applications, it lacks consideration of serverless-specific factors such as cold vs. warm starts. Lumos captures the trade-offs of using WebAssembly in serverless computing in heterogeneous environments such as the Edge-Cloud Continuum rather than standalone Wasm runtimes in isolation. In [16], authors investigate the performance implications of integrating Wasm into container runtimes across diverse hardware platforms, emphasizing image size, startup time, and container pull time, but do not consider workload characteristics, such as workload data IO, serialization, or WASI overhead. In [29], the authors present a comprehensive benchmark of Wasm for embedded systems, analyzing performance, memory use, and runtime features. They show

that WASM, especially with AoT compilation, offers competitive performance and memory efficiency. Liu et. al. [21] evaluate the performance of Wasm-based container runtimes in comparison to traditional Docker containers and standalone Wasm runtimes. Liu shows that Wasm containers currently fail to deliver performance advantages over Docker containers due to overheads introduced by WASI. However, Liu’s benchmark only considers Wasm in interpreter mode, which suffers from performance overhead. In Lumos, we identify the WASI overhead in interpreter and in AoT compilation, providing insightful data on how to improve performance in cases where WASI overhead is a bottleneck. In [11], the authors propose an execution model for serverless functions at the edge by replacing containers with Wasm modules. Although authors show that wasm-based serverless functions suffer performance overhead compared to native containers, they do not identify application characteristics that directly impact performance.

In contrast to the presented state-of-the-art approaches, Lumos evaluates Wasm and container runtimes across diverse workload types (CPU- and I/O-bound) under varying invocation patterns and in heterogeneous environments, also capturing the performance overhead from workloads that leverage WASI, enabling a more comprehensive performance characterization of WebAssembly as a serverless runtime in the Edge-Cloud Continuum.

8 Conclusion & Future Work

In this paper, we introduced Lumos, a benchmarking tool and performance model for evaluating serverless runtimes in the Edge-Cloud Continuum. Lumos provides a performance characterization of serverless workloads with different characteristics, such as compute- and data-intensive, by analyzing fine-grained metrics such as total function execution time, cold and warm starts, as well as IO and serialization latency. Our experiments provide a benchmark of Wasm and container as serverless runtimes, evaluating their performance based on workload, environment, and system factors. Our results show that Wasm still incurs notable latency overhead, especially in interpreted mode, where latency can increase by up to 27x compared to containers. Additionally, we further observe that Wasm incurs up to 7x higher I/O and up to 10x higher serialization overhead, posing significant challenges for data-intensive workloads, which can limit Wasm’s benefits in such scenarios.

In the future, we plan to extend Lumos to enable the performance characterization of serverless functions with real-world traces and in different environments, such as the 3D Computing Continuum that unifies Edge-Cloud and Space. Lumos will allow users to identify performance bottlenecks and evaluate function trade-offs in edge and orbital environments without requiring real satellite deployments.

Acknowledgments

This work is partially funded by the Austrian Research Promotion Agency (FFG) under the project RapidREC (Project No. 903884). This work has received funding from the Austrian Internet Stiftung under the NetIdee project LEO Trek (ID 7442). Partly funded by EU Horizon Europe under GA no. 101070186 (TEADAL) and GA no. 101192912 (NexaSphere).

References

- [1] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, et al. 2018. SAND: Towards High-Performance Serverless Computing. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 923–935.
- [2] Marcin Copik, Alexandru Calotiu, and Torsten Hoefler. 2025. SeBS 2.0: Keeping up with the Clouds. In *Proceedings of the 3rd Workshop on Serverless Systems, Applications and Methodologies* (Rotterdam, Netherlands) (SESAME' 25). 42–44. doi:10.1145/3721465.3721867
- [3] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, et al. 2021. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. In *Proceedings of the 22nd International Middleware Conference* (Québec city, Canada) (Middleware '21). 64–78. doi:10.1145/3464298.3476133
- [4] Datadog. 2023. The State of Serverless. <https://www.datadoghq.com/state-of-serverless/>.
- [5] Edgerun. 2025. TelemD. <https://github.com/edgerun/telemD>
- [6] Simon Eismann, Joel Scheuner, Erwin van Eyk, et al. 2022. The State of Serverless Applications: Collection, Characterization, and Community Consensus. *IEEE Transactions on Software Engineering* 48, 10 (2022), 4152–4166. doi:10.1109/TSE.2021.3113940
- [7] Simon Eismann, Joel Scheuner, Erwin van Eyk, et al. 2021. Serverless Applications: Why, When, and How? *IEEE Software* 38, 1 (2021), 32–39. doi:10.1109/MS.2020.3023302
- [8] Philipp Gackstatter, Pantelis A. Frangoudis, and Schahram Dustdar. 2022. Pushing Serverless to the Edge with WebAssembly Runtimes. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 140–149. doi:10.1109/CCGrid54584.2022.00023
- [9] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, et al. 2020. Sledge: a Serverless-first, Light-weight Wasm Runtime for the Edge. In *Proceedings of the 21st International Middleware Conference* (Delft, Netherlands) (Middleware '20). 265–279. doi:10.1145/3423211.3425680
- [10] Phani Kishore Gadepalli, Gregor Peach, Ludmila Cherkasova, Rob Aitken, and Gabriel Parmer. 2019. Challenges and Opportunities for Efficient Serverless Computing at the Edge. In *2019 38th Symposium on Reliable Distributed Systems (SRDS)*. 261–2615. doi:10.1109/SRDS47363.2019.00036
- [11] Adam Hall and Umakishore Ramachandran. 2019. An execution model for serverless functions at the edge. In *Proceedings of the International Conference on Internet of Things Design and Implementation* (Montreal, Quebec, Canada) (IoTDI '19). 225–236. doi:10.1145/3302505.3310084
- [12] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. 2019. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 107–120.
- [13] Shuyao Jiang, Ruiying Zeng, Zihao Rao, et al. 2024. Revealing Performance Issues in Server-Side WebAssembly Runtimes via Differential Testing. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) (ASE '23). IEEE Press, 661–672. doi:10.1109/ASE56229.2023.00088
- [14] Eric Jonas, Johann Schleier-Smith, et al. 2019. Cloud Programming Simplified: A Berkeley View on Serverless Computing. arXiv:1902.03383 [cs.OS]
- [15] Sangeeta Kakati and Mats Brorsson. 2024. A Cross-Architecture Evaluation of WebAssembly in the Cloud-Edge Continuum. In *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 337–346. doi:10.1109/CCGrid59990.2024.00046
- [16] Sangeeta Kakati and Mats Brorsson. 2025. Performance and Usability Implications of Multiplatform and WebAssembly Containers. In *Proceedings of the 15th International Conference on Cloud Computing and Services Science - Volume 1: CLOSER, INSTICC, SciTePress*, 15–25. doi:10.5220/001320320300003950
- [17] Vojdan Kjorveziroski and Sonja Filiposka. 2023. WebAssembly as an Enabler for Next Generation Serverless Computing. *Journal of Grid Computing* 21, 3 (2023), 34. doi:10.1007/s10723-023-09669-8
- [18] Vojdan Kjorveziroski and Sonja Filiposka. 2023. WebAssembly Orchestration in the Context of Serverless Computing. *Journal of Network and Systems Management* 31, 3 (2023), 62. doi:10.1007/s10922-023-09753-0
- [19] Swaroop Kotni, Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. Faast-lane: Accelerating Function-as-a-Service Workflows. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 805–820.
- [20] Varad Kulkarni, Nikhil Reddy, Tuhin Khare, et al. 2024. XFBench: A Cross-Cloud Benchmark Suite for Evaluating FaaS Workflow Platforms. In *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 543–556. doi:10.1109/CCGrid59990.2024.00067
- [21] Mugeng Liu, Haiyang Shen, Yixuan Zhang, et al. 2025. WebAssembly for Container Runtime: Are We There Yet? *ACM Trans. Softw. Eng. Methodol.* (Feb. 2025). doi:10.1145/3712197
- [22] Qingyuan Liu, Dong Du, Yubin Xia, Ping Zhang, and Haibo Chen. 2023. The Gap Between Serverless Research and Real-World Systems. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) (SoCC '23). 475–485. doi:10.1145/3620678.3624785
- [23] Fangming Lu, Xingda Wei, Zhuobin Huang, Rong Chen, Minyu Wu, and Haibo Chen. 2024. Serialization/Deserialization-free State Transfer in Serverless Workflows. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (EuroSys '24). Association for Computing Machinery, New York, NY, USA, 132–147. doi:10.1145/3627703.3629568
- [24] Cynthia Marcelino, Leonard Guelmino, Thomas Pusztai, and Stefan Nastic. 2025. Databelt: A continuous data path for serverless workflows in the 3D compute continuum. *Journal of Systems Architecture* (2025). doi:10.1016/j.sysarc.2025.103577
- [25] Cynthia Marcelino and Stefan Nastic. 2024. CWASI: A WebAssembly Runtime Shim for Inter-function Communication in the Serverless Edge-Cloud Continuum. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing* (Wilmington, DE, USA) (SEC '23). 158–170. doi:10.1145/3583740.3626611
- [26] Cynthia Marcelino and Stefan Nastic. 2024. Truffle: Efficient Data Passing for Data-Intensive Serverless Workflows in the Edge-Cloud Continuum. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. 53–62. doi:10.1109/UCC63386.2024.00017
- [27] Cynthia Marcelino, Jack Shahhoud, and Stefan Nastic. 2025. GoldFish: Serverless Actors with Short-Term Memory State for the Edge-Cloud Continuum. In *Proceedings of the 14th International Conference on the Internet of Things (IoT '24)*. 56–64. doi:10.1145/3703790.3703797
- [28] Pankaj Mendki. 2020. Evaluating Webassembly Enabled Serverless Approach for Edge Computing. In *2020 IEEE Cloud Summit*. 161–166. doi:10.1109/IEEECloudSummit48914.2020.00031
- [29] Konrad Moron and Stefan Wallentowitz. 2025. Benchmarking WebAssembly for Embedded Systems. *ACM Trans. Archit. Code Optim.* (May 2025). doi:10.1145/3736169
- [30] Jāmes Ménétrey, Marcelo Pasin, Pascal Felber, and Valerio Schiavoni. 2022. WebAssembly as a Common Layer for the Cloud-edge Continuum. In *Proceedings of the 2nd Workshop on Flexible Resource and Application Management on the Edge* (Minneapolis, MN, USA) (FRAME '22). Association for Computing Machinery, New York, NY, USA, 3–8. doi:10.1145/3526059.3533618
- [31] Stefan Nastic. 2024. Self-Provisioning Infrastructures for the Next Generation Serverless Computing. *SN Computer Science* 5, 6 (2024), 678 – 693. doi:10.1007/s42979-024-03022-w
- [32] Stefan Nastic, Andrea Morichetta, Thomas Pusztai, Schahram Dustdar, Xiaoning Ding, Deepak Vij, and Ying Xiong. 2020. Sloc: Service level objectives for next generation cloud computing. *IEEE Internet Computing* 24, 3 (2020), 39–50.
- [33] Stefan Nastic, Philipp Raith, Alireza Furutanpey, Thomas Pusztai, and Schahram Dustdar. 2022. A Serverless Computing Fabric for Edge & Cloud. In *2022 IEEE 4th International Conference on Cognitive Machine Intelligence (CogMI)*. 1–12. doi:10.1109/CogMI56440.2022.00011
- [34] Thomas Pusztai, Stefan Nastic, Andrea Morichetta, et al. 2022. Polaris scheduler: SLO- and topology-aware microservices scheduling at the edge. In *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 61–70.
- [35] Kaustubh Rajendra Rajput, Chinmay Dilip Kulkarni, et al. 2022. EdgeFaaS Bench: Benchmarking Edge Devices Using Serverless Computing. In *2022 IEEE International Conference on Edge Computing and Communications (EDGE)*. 93–103. doi:10.1109/EDGE55608.2022.00024
- [36] Johann Schleier-Smith, Vikram Sreekanti, et al. 2021. What Serverless Computing is and Should Become: The next Phase of Cloud Computing. *Commun. ACM* 64, 5 (apr 2021), 76–84. doi:10.1145/3406011
- [37] Larissa Schmid, Marcin Copik, et al. 2025. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (EuroSys '25). 902–920. doi:10.1145/3689031.3717465
- [38] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 419–433.
- [39] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. 2018. Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 133–146.
- [40] Wenwen Wang. 2022. How Far We've Come – A Characterization Study of Standalone WebAssembly Runtimes. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 228–241. doi:10.1109/IISWC55918.2022.00028
- [41] Jinfeng Wen, Zhenpeng Chen, Federica Sarro, and Shangguang Wang. 2025. Unveiling overlooked performance variance in serverless computing. *Empirical Software Engineering* 30, 2 (2025), 1–26.
- [42] Miguel G Xavier, Israel C De Oliveira, Fabio D Rossi, et al. 2015. A performance isolation analysis of disk-intensive workloads on container-based clouds. In *2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. IEEE, 253–260.
- [43] Dong Xie, Yang Hu, and Li Qin. 2021. An evaluation of serverless computing on x86 and arm platforms: Performance and design implications. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE, 313–321.
- [44] Li Zhou, Yifan Zhang, Youhuizi Li, Na Yun, and Lifeng Yu. 2017. I/O Performance Isolation Analysis and Optimization on Linux Containers. In *SEKE*. 61–66.